

std::basic_fixed_string

Document #: P3094R0
Date: 2024-01-18
Project: Programming Language C++
Audience: SG16 Unicode
SG18 Library Evolution Working Group Incubator (LEWGI)
Library Evolution Working Group
Reply-to: Mateusz Pusz ([Epam Systems](#))
<mateusz.pusz@gmail.com>

Contents

- [1 Introduction](#)
- [2 fixed_string is an established practice](#)
- [3 Minimal interface requirements](#)
- [4 Interface](#)
- [5 Implementation experience](#)
- [6 fixed_string design alternatives](#)
 - [6.1 Full string_view-like interface](#)
 - [6.2 Mutation interface](#)
 - [6.3 inplace_string](#)
 - [6.4 std::string with a static storage allocator](#)
 - [6.5 Just wait for the C++ language to solve it](#)
- [7 Acknowledgements](#)
- [8 References](#)

§ 1 Introduction

This paper proposes standardizing a string type that could be used at compile-time as a non-type template parameter (NTTP). This means that such string type has to satisfy the structural type requirements of the C++ language. One of such requirements is to expose all the data members publicly. So far, none of the existing string types in the C++ standard library satisfies such requirements.

This type is intended to serve as an essential building block and be exposed as a part of the public interface of quantities and units library proposed in [P2980R1]. All dimensions, units, prefixes, and constants definitions will use it to provide their textual representation.

§ 2 `fixed_string` is an established practice

There are plenty of home-grown `fixed_string` types. Every project that wants to pass text as NTTP already provides its own version of it. There are also some that predate C++20 and do not satisfy structural type requirements.

A [quick search in GitHub](#) returns plenty of results. Let's mention a few of those:

- [mpusz/mp-units](#)
- [fmtlib/fmt](#)
- [hanickadot/compile-time-regular-expressions](#)
- [tomazos/fixed_string](#) - a reference implementation of [P0259R0] (not a structural type as it is 8 years old)
- [unterumarmung/fixed_string](#)
- [mu001999/fixed_string](#)
- [calebxyz/Fixed_Strings](#)
- [astral-shining/FixedString](#)
- [ynsn/fixed_string](#)

Having such a type in the C++ standard library will prevent reinventing the wheel by the community and reimplementing it in every project that handles text at compile-time.

§ 3 Minimal interface requirements

Such type should at least:

- satisfy structural type requirements,
- be equality comparable and potentially totally ordered,
- store and provide concatenation support for zero-ended strings,
- provide storage that, if set at compile time, would also be available for read-only access at runtime,
- provide at least read-only access to the contained storage.

It does not need to expose a string-like interface. In case its interface is immutable, as it is proposed by the author, we can easily wrap it with `std::string_view` to get such an

interface for free.

§ 4 Interface

`fixed_string` is a read-only text container. It satisfies structural type requirements, its size is fixed as one of the class template parameters, and it has a minimalistic range/view-like interface:

```
template<typename CharT, std::size_t N>
struct basic_fixed_string {
    CharT data_[N + 1] = {}; // exposition only

    using value_type = CharT;
    using pointer = CharT*;
    using const_pointer = const CharT*;
    using reference = CharT&;
    using const_reference = const CharT&;
    using const_iterator = const CharT*;
    using iterator = const_iterator;
    using size_type = std::size_t;
    using difference_type = std::ptrdiff_t;

    constexpr explicit(false) basic_fixed_string(const CharT (&txt)[N + 1])
        noexcept;
    constexpr basic_fixed_string(const CharT* ptr,
        std::integral_constant<std::size_t, N>) noexcept;

    template<std::convertible_to<CharT>... Rest>
    requires(1 + sizeof...(Rest) == N)
    constexpr explicit basic_fixed_string(CharT first, Rest... rest)
        noexcept;

    [[nodiscard]] constexpr bool empty() const noexcept;
    [[nodiscard]] constexpr size_type size() const noexcept;
    [[nodiscard]] constexpr const_pointer data() const noexcept;
    [[nodiscard]] constexpr const CharT* c_str() const noexcept;
    [[nodiscard]] constexpr value_type operator[](size_type index) const
        noexcept;

    [[nodiscard]] constexpr const_iterator begin() const noexcept;
    [[nodiscard]] constexpr const_iterator cbegin() const noexcept;
    [[nodiscard]] constexpr const_iterator end() const noexcept;
    [[nodiscard]] constexpr const_iterator cend() const noexcept;

    [[nodiscard]] constexpr std::basic_string_view<CharT> view() const
        noexcept;

    template<std::size_t N2>
    [[nodiscard]] constexpr friend basic_fixed_string<CharT, N + N2>
        operator+(const basic_fixed_string& lhs,

            const basic_fixed_string<CharT, N2>& rhs) noexcept;
```

```

[[nodiscard]] constexpr bool operator==(const basic_fixed_string& other)
    const;
template<std::size_t N2>
    [[nodiscard]] friend constexpr bool operator==(const
        basic_fixed_string&, const basic_fixed_string<CharT, N2>&);

template<std::size_t N2>
    [[nodiscard]] friend constexpr auto operator<==(const
        basic_fixed_string& lhs,
                                                    const
        basic_fixed_string<CharT, N2>& rhs);

template<typename Traits>
    friend std::basic_ostream<CharT, Traits>&
        operator<<(std::basic_ostream<CharT, Traits>& os,
                                                    const
        basic_fixed_string<CharT, N>& str);
};

template<typename CharT, std::size_t N>
basic_fixed_string(const CharT (&str)[N]) -> basic_fixed_string<CharT, N -
    1>;

template<typename CharT, std::size_t N>
basic_fixed_string(const CharT* ptr, std::integral_constant<std::size_t,
    N>) -> basic_fixed_string<CharT, N>;

template<typename CharT, std::convertible_to<CharT>... Rest>
basic_fixed_string(CharT, Rest...) -> basic_fixed_string<CharT, 1 +
    sizeof...(Rest)>;

template<std::size_t N>
using fixed_string = basic_fixed_string<char, N>;

template<typename CharT, std::size_t N>
struct std::formatter<basic_fixed_string<CharT, N>> :
    formatter<std::basic_string_view<CharT>> {
    template<typename FormatContext>
    auto format(const basic_fixed_string<CharT, N>& str, FormatContext& ctx)
    {
        return formatter<std::basic_string_view<CharT>>::format(str.view(),
            ctx);
    }
};

```

Please note that there are nearly no text-specific member functions inside (besides `.c_str()` that could be easily omitted and replaced with `.data()` in the user's code).

This type could probably even serve as a generic storage structural type if it didn't have an invariant requiring its internal contiguous data storage to be of size `N + 1` and ending with `\0`.

§ 5 Implementation experience

This particular interface is implemented and successfully used in the [mp-units](#) project.

§ 6 **fixed_string design alternatives**

§ 6.1 **Full string_view-like interface**

We could add a whole `string_view`-like interface to this class, but the author decided not to do it. This will add plenty of overloads that will probably not be used too often by the users of this particular type anyway. Doing that will add a maintenance burden to keep it consistent with all other string-like types in the library.

If the user needs to obtain a `string_view`-like interface to work with this type, then `std::string_view` itself can easily be used to achieve that:

```
basic_fixed_string txt = "abc";  
auto pos = txt.view().find_first_of('b');
```

§ 6.2 **Mutation interface**

As stated above, this type is read-only. As we can't resize the string, it could be considered controversial to allow mutation of the contents. The only thing we could provide would be support for overwriting specific characters in the internal storage. This, however, does not seem to be a common use case for such a type.

If passed as an NTTP, such type would be `const` at runtime, which would disable all the mutation interface anyway.

On the other hand, such an interface would allow running every `constexpr` algorithm from the C++ standard library on such a range at compile time.

If we decide to add such an interface, it is worth pointing out that wrapping the type in the `std::string_view` would not be enough to obtain a proper string-like mutating interface. As we do not have another string-like reference type that provides mutating capabilities we can end up with the need to implement the entire string interface in this class as well.

This is why the author does not propose to add it in the first iteration. We can always easily add such an interface later if a need for it arises.

§ 6.3 **inplace_string**

We could also consider providing a full-blown fixed-capacity string class similar to the `inplace_vector` [P0843R9]. It would provide not only full read-write capability but also be really beneficial for embedded and low-latency domains.

Despite being welcomed and valuable in the C++ community, the author believes that such a type should not satisfy the current requirements for a structural type, which is a hard requirement of this use case. If `inplace_string` was used instead, we would end up

with separate template instantiations for objects with the same value but a different capacity. We prefer to prevent such a behavior.

§ 6.4 `std::string` with a static storage allocator

We could also try to use `std::string` with a static storage allocator, but this solution does not meet structural type requirements and could be an overkill for this use case, resulting with significantly decreased compile times.

Also, it would be hard or impossible to make the storage created at compile-time to be accessible at runtime in such cases.

§ 6.5 Just wait for the C++ language to solve it

[P2484R0] proposed extending support for class types as non-type template parameters in the C++ language. However, this proposal's primary author is no longer active in C++ standardization, and there have been no updates to the paper in the last two years.

We can't wait for the C++ language to change forever. This library will be impossible to standardize without such a feature. This is why the author recommends progressing with the `fixed_string` approach.

§ 7 Acknowledgements

Special thanks and recognition goes to [Epam Systems](#) for supporting Mateusz's membership in the ISO C++ Committee and the production of this proposal.

The author would also like to thank Hana Dusíková for providing valuable feedback that helped him shape the final version of this document.

§ 8 References

[P0259R0] Michael Price, Andrew Tomazos. 2016-02-12. `fixed_string`: a compile-time string.

<https://wg21.link/p0259r0>

[P0843R9] Gonzalo Brito Gadeschi, Timur Doumler, Nevin Liber, David Sankel. 2023-09-14. `inplace_vector`.

<https://wg21.link/p0843r9>

[P2484R0] Richard Smith. 2021-11-17. Extending class types as non-type template parameters.

<https://wg21.link/p2484r0>

[P2980R1] Mateusz Pusz, Dominik Berner, Johel Ernesto Guerrero Peña, Charles Hogg, Nicolas Holthaus, Roth Michaels, Vincent Reverdy. 2023-11-28. A motivation, scope, and plan for a quantities and units library.

<https://wg21.link/p2980r1>